📡 信贷流量平台 API 接口文档

一、接口概览

1.1 基础信息

项目	值
基础URL	`https://api.example.com`
数据格式	JSON
编码	UTF-8
签名算法	MD5

1.2 接口列表

上游接口（外部调用我们）

接口	方法	说明
/api/v1/auth/register	POST	上游注册
/api/v1/auth/login	POST	上游登录
/api/v1/inflow/push	POST	推送线索
/api/v1/inflow/batch	POST	批量推送线索

接口	方法	说明
/api/v1/channel/info	GET	渠道信息
/api/v1/channel/stats	GET	渠道统计
/api/v1/channel/bill	GET	账单明细

下游接口（外部调用我们）

接口	方法	说明
/api/v1/client/register	POST	下游注册
/api/v1/client/login	POST	下游登录
/api/v1/client/callback	POST	接收线索回调
/api/v1/client/callback/config	POST	配置回调地址
/api/v1/client/stats	GET	推送统计
/api/v1/client/balance	GET	账户余额

二、上游 API

2.1 上游注册

```
POST /api/v1/auth/register
```

请求体

```
{
  "username": "channel_a",
  "password": "password123",
  "company": "渠道A公司",
  "contact": "张经理",
  "phone": "13800138000",
  "email": "zhang@example.com"
}
```

响应

```
{
  "code": 0,
  "message": "success",
```

```
  "data": {
    "appId": "uk_abc123",
    "appSecret": "secret_xyz789",
    "username": "channel_a"
  }
}
```

2.2 上游登录

```
POST /api/v1/auth/login
```

请求体

```
{
  "username": "channel_a",
  "password": "password123"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 7200
  }
}
```

2.3 推送线索

```
POST /api/v1/inflow/push
```

请求头

参数	说明
Authorization	Bearer {token}
X-Signature	MD5签名

请求体

```
{
  "phone": "13800138000",
  "name": "张三",
  "amount": "50000",
  "area": "北京",
  "gender": "男",
  "age": 35,
  "income": "20000",
  "remark": "备注信息"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "leadId": "LD20240320001",
    "phone": "13800138000",
    "receiveTime": "2024-01-15T14:30:00Z"
  }
}
```

2.4 批量推送线索

```
POST /api/v1/inflow/batch
```

请求体

```
{
  "leads": [
    {
      "phone": "13800138000",
      "name": "张三",
      "amount": "50000"
    },
    {
      "phone": "13900139000",
      "name": "李四",
      "amount": "30000"
    }
  ]
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "total": 2,
    "success": 2,
    "failed": 0,
    "results": [
      { "phone": "13800138000", "leadId": "LD20240320001", "status": "success" },
      { "phone": "13900139000", "leadId": "LD20240320002", "status": "success" }
    ]
  }
}
```

2.5 渠道信息

```
GET /api/v1/channel/info
```

响应

```
{
  "code": 0,
  "data": {
    "appId": "uk_abc123",
    "company": "渠道A公司",
    "contact": "张经理",
    "phone": "13800138000",
    "status": "active",
    "createTime": "2024-01-01T00:00:00Z"
  }
}
```

2.6 渠道统计

```
GET /api/v1/channel/stats?date=2024-01-15
```

响应

```
{
  "code": 0,
  "data": {
    "today": {
      "pushCount": 156,
```

```
      "successCount": 152,
      "failCount": 4
    },
    "month": {
      "pushCount": 4560,
      "successCount": 4480,
      "failCount": 80
    }
  }
}
```

2.7 账单明细

```
GET /api/v1/channel/bill?startDate=2024-01-01&endDate=2024-01-15
```

响应

```
{
  "code": 0,
  "data": {
    "list": [
      {
        "date": "2024-01-15",
        "quantity": 156,
        "unitPrice": 5.0,
        "amount": 780.0,
        "status": "pending"
      }
    ],
    "total": 4560.0,
    "pending": 2340.0,
    "paid": 2220.0
  }
}
```

三、下游 API

3.1 下游注册

```
POST /api/v1/client/register
```

请求体

```
{
  "username": "client_zhang",
  "password": "password123",
  "company": "客户张三公司",
  "contact": "李总",
  "phone": "13900139000",
  "email": "li@example.com"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "clientId": "client_001",
    "apiKey": "sk_live_abc123****xyz",
    "username": "client_zhang"
  }
}
```

3.2 下游登录

```
POST /api/v1/client/login
```

请求体

```
{
  "username": "client_zhang",
  "password": "password123"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 7200
  }
}
```

3.3 配置回调地址

```
POST /api/v1/client/callback/config
```

请求体

```
{
  "callbackUrl": "https://your-server.com/webhook",
  "callbackSecret": "your_webhook_secret"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "callbackUrl": "https://your-server.com/webhook",
    "status": "active"
  }
}
```

3.4 接收线索回调

这是我们主动调用的接口，下游需实现

```
POST 下游回调地址
```

请求头

参数	说明
Content-Type	application/json
X-Signature	MD5签名

请求体

```
{
  "leadId": "LD20240320001",
  "phone": "13800138000",
  "name": "张三",
  "amount": "50000",
  "area": "北京",
```

```
  "gender": "男",
  "age": 35,
  "income": "20000",
  "source": "渠道A",
  "pushTime": "2024-01-15T14:30:05Z"
}
```

下游响应

```
{
  "code": 0,
  "message": "success"
}
```

3.5 推送统计

```
GET /api/v1/client/stats?date=2024-01-15
```

响应

```
{
  "code": 0,
  "data": {
    "today": {
      "receivedCount": 45,
      "successCount": 42,
      "failCount": 3
    },
    "month": {
      "receivedCount": 1350,
      "successCount": 1290,
      "failCount": 60
    }
  }
}
```

3.6 账户余额

```
GET /api/v1/client/balance
```

响应

```
{
  "code": 0,
  "data": {
    "balance": 5000.0,
    "freeze": 1000.0,
    "available": 4000.0,
    "todayCost": 450.0
  }
}
```

四、错误码

code	message	说明
0	success	成功
1001	missing parameter	缺少必填参数
1002	invalid phone	手机号格式错误
1003	signature error	签名错误
1004	channel disabled	渠道已禁用
1005	rate limit	请求过于频繁
2001	unauthorized	未授权
2002	token expired	Token过期
3001	callback failed	回调失败
3002	insufficient balance	余额不足

五、签名算法

5.1 生成签名

```
Signature = MD5(JSON.stringify(body) + Secret)
```

Node.js 示例

```
const crypto = require('crypto');
```

```
function generateSignature(body, secret) {
  const signString = JSON.stringify(body) + secret;
  return crypto.createHash('md5').update(signString).digest('hex');
}
```

Python 示例

```
import hashlib
import json

def generate_signature(body, secret):
    sign_string = json.dumps(body, ensure_ascii=False) + secret
    return hashlib.md5(sign_string.encode('utf-8')).hexdigest()
```

六、Demo 代码

6.1 上游推送（Node.js）

```
const axios = require('axios');
const crypto = require('crypto');

const CONFIG = {
  API_URL: 'https://api.example.com',
  APP_ID: 'your_app_id',
  APP_SECRET: 'your_app_secret'
};

function sign(body, secret) {
  return crypto.createHash('md5')
    .update(JSON.stringify(body) + secret).digest('hex');
}

async function login() {
  const res = await axios.post(`${CONFIG.API_URL}/api/v1/auth/login`, {
    username: CONFIG.APP_ID,
    password: 'password123'
  });
  return res.data.data.token;
}

async function pushLead(token, phone, name) {
  const body = { phone, name, amount: '50000', area: '北京' };
  const headers = {
    'Authorization': `Bearer ${token}`,
    'X-Signature': sign(body, CONFIG.APP_SECRET)
  };
  return axios.post(`${CONFIG.API_URL}/api/v1/inflow/push`, body, { headers });
```

```
}

async function main() {
  const token = await login();
  const result = await pushLead(token, '13800138000', '张三');
  console.log(result.data);
}

main();
```

6.2 下游接收（Node.js）

```
const express = require('express');
const crypto = require('crypto');
const app = express();

app.use(express.json());

const WEBHOOK_SECRET = 'your_webhook_secret';

app.post('/webhook', (req, res) => {
  const sig = req.headers['x-signature'];
  const expected = crypto.createHash('md5')
    .update(JSON.stringify(req.body) + WEBHOOK_SECRET).digest('hex');

  if (sig !== expected) {
    return res.status(401).json({ code: 1003, message: 'signature error' });
  }

  const { leadId, phone, name, amount } = req.body;
  console.log('收到线索:', { leadId, phone, name, amount });

  res.json({ code: 0, message: 'success' });
});

app.listen(3000, () => console.log('服务启动: 3000'));
```

七、测试

测试账号

角色	用户名	密码
上游	test_upstream	test123456
下游	test_downstream	test123456

测试命令

```
# 启动本地 Mock Server
node server/mock-api.js

# 测试登录
curl -X POST http://localhost:3000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"test_upstream","password":"test123456"}'
```

八、注意事项

事项	说明
请求频率	不超过 10 次/秒
Token有效期	2小时
签名密钥	请妥善保管
重试机制	失败自动重试 3 次
数据安全	敏感信息建议加密

文档版本: v1.0
更新时间: 2024-01-15